

ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No.34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

Database Management Systems

AIRLINE RESERVATION SYSTEM

A Relational Database Design for Flight Booking & Management

Bachelor of Computer Application
Semester – III

R MONISHA BANGERA
2511021240053

Submitted To -

Faculty Name: Renita Joy R

Faculty Signature:

Advanced SQL Queries Using Joins and Subqueries

1. Introduction

This report presents 18 advanced queries on the Airline Reservation System database, loaded with sample_data_insertion.sql (20 passengers, 12 airports, 8 aircraft, 28 flights, 50 bookings). The queries use joins to combine related tables and subqueries to answer questions that need a result inside another query. Together they produce reports an airline would actually use: passenger manifests, flight load, revenue, customer value, airport traffic and a management summary. Every query is followed by its output and a short explanation, and all of them are in the script advanced_queries.sql.

Techniques covered

Technique	Used in
INNER JOIN across several tables	Q1, Q2 and the reports Q6 to Q9
LEFT JOIN with IS NULL (find missing related records)	Q3, Q4
Self join	Q5
Joins combined with GROUP BY and aggregates	Q6 to Q9
Subquery in FROM (derived table)	Q10, Q16
Scalar subquery	Q11, Q17, Q18
Subquery with IN	Q12
Correlated subquery with EXISTS, NOT EXISTS and MAX	Q13, Q14, Q15

2. Joins

Q1. Passenger manifest for flight AI502 on 1 November (INNER JOIN, 5 tables)

```
SELECT b.Booking_ID,
       CONCAT(p.First_Name, ' ', p.Last_Name) AS Passenger,
       s.Seat_Number, s.Seat_Class,
       CONCAT(dep.Airport_Code, ' -> ', arr.Airport_Code) AS Route,
       b.Booking_Status, b.Fare
FROM Booking b
JOIN Passenger p ON p.Passenger_ID = b.Passenger_ID
JOIN  Seat  JOIN   s ON s.Seat_ID              = b.Seat_ID
Flight      JOIN   f ON f.Flight_ID            = b.Flight_ID
Airport     JOIN   dep ON dep.Airport_ID = f.Departure_Airport_ID
Airport            arr ON arr.Airport_ID = f.Arrival_Airport_ID
WHERE f.Flight_ID = 101
ORDER BY s.Seat_Class, s.Seat_Number;
```

Booking_ID	Passenger	Seat_Number	Seat_Class	Route BLR	Booking_Status	Fare
5003	Priya Nair	2A	Business	-> DEL BLR	Confirmed	13800.0
5001	Monisha Bangera	12A	Economy	-> DEL BLR	Confirmed	0
5002	Rahul Sharma	12B	Economy	-> DEL BLR	Confirmed	5500.00
5004	Arjun Reddy	18C	Economy	-> DEL BLR	Confirmed	5500.00
5005	Ananya Iyer	18D	Economy	-> DEL	Cancelled	5200.00

5200.00

Explanation: An INNER JOIN returns only rows that match in every table. Airport is joined twice under two aliases (dep and arr) because Flight has two foreign keys to it. This is the manifest for the flight; the cancelled booking 5005 still appears, so a boarding list would add WHERE b.Booking_Status = 'Confirmed'.

Q2. Complete travel record of one passenger across all nine tables

```
SELECT CONCAT(p.First_Name, ' ', p.Last_Name) AS Passenger,
       f.Flight_Number, f.Departure_DateTime,
       CONCAT(dep.Airport_Code, ' -> ', arr.Airport_Code) AS Route,
       a.Aircraft_Model, s.Seat_Number, s.Seat_Class,
       t.Ticket_Number, t.Ticket_Status,
       pay.Payment_Method, pay.Amount, pay.Payment_Status,
       g.Baggage_Count, g.Total_Weight
FROM Passenger p
    JOIN Booking b    ON b.Passenger_ID = p.Passenger_ID
JOIN Flight      f    ON f.Flight_ID    = b.Flight_ID
     JOIN Aircraft a  ON a.Aircraft_ID  = f.Aircraft_ID
JOIN Airport dep ON dep.Airport_ID = f.Departure_Airport_ID
JOIN Airport arr ON arr.Airport_ID = f.Arrival_Airport_ID
JOIN Seat        s    ON s.Seat_ID      = b.Seat_ID
LEFT JOIN Ticket t        ON t.Booking_ID   = b.Booking_ID
LEFT JOIN Payment pay ON pay.Booking_ID = b.Booking_ID AND pay.Payment_Status
= 'Successful'
LEFT JOIN Baggage g       ON g.Booking_ID   = b.Booking_ID
WHERE p.Passenger_ID = 18
ORDER BY f.Departure_DateTime;
```

Passenger	Flight_Number	Departure_DateTime	Route	Aircraft_Model	Seat_Number	Seat_Class	Ticket_Numbe r	Ticket_Status	Payment_Method	Amount	Payment_Status	Baggage_Count	Total_Weight
Aarav Patel	UK821	2026-11-01 07:15:00	BOM -> DEL	Airbus A321	1A	First	TKT-UK821-05023	Valid	Card	21500.00	Successful	3	45.00
Aarav Patel	UK822	2026-11-01 11:00:00	DEL -> BOM	Airbus A321	1A	First	TKT-UK822-05027	Valid	Net Banking	21800.00	Successful	3	46.50
Aarav Patel	AI346	2026-11-03 23:55:00	BLR -> SIN	Airbus A320neo	3A	Business	TKT-AI346-05047	Valid	Card	38500.00	Successful	2	34.00

Explanation: Gathers every record related to passenger 18 (Aarav Patel). INNER JOINs follow the mandatory relationships; LEFT JOINs are used for Ticket, Payment and Baggage because a booking may not have them yet, and the booking must still appear.

Q3. Flights that nobody has booked (LEFT JOIN ... IS NULL)

```
SELECT f.Flight_ID, f.Flight_Number, f.Departure_DateTime,
       CONCAT(dep.Airport_Code, ' -> ', arr.Airport_Code) AS Route,
       f.Flight_Status
FROM Flight f
JOIN Airport dep ON dep.Airport_ID = f.Departure_Airport_ID
JOIN Airport arr ON arr.Airport_ID = f.Arrival_Airport_ID
LEFT JOIN Booking b ON b.Flight_ID = f.Flight_ID
WHERE b.Booking_ID IS NULL
ORDER BY f.Departure_DateTime, f.Flight_ID;
```

Flight_ID	Flight_Number	Departure_DateTime	Route	Flight_Status
118	IX772	2026-11-01 09:30:00	COK -> BLR	Scheduled
123	6E411	2026-11-01 13:00:00	MAA -> BLR	Scheduled
119	IX771	2026-11-02 07:30:00	BLR -> COK	Scheduled
116	SG145	2026-11-02 08:00:00	DEL -> CCU	Scheduled
109	6E204	2026-11-02 09:30:00	BLR -> BOM	Scheduled
127	UK955	2026-11-02 14:00:00	DEL -> HYD	Scheduled
122	AI934	2026-11-03 02:30:00	BLR -> DXB	Scheduled
106	AI503	2026-11-03 10:00:00	DEL -> BLR	Scheduled

Explanation: A LEFT JOIN keeps every flight, filling the Booking columns with NULL when no booking exists. Filtering on NULL then leaves only flights with no bookings: candidates for promotion or cancellation.

Q4. Bookings that have no ticket yet, with their payment status

```
SELECT b.Booking_ID,
       CONCAT(p.First_Name, ' ', p.Last_Name) AS Passenger,
       f.Flight_Number, b.Booking_Status, b.Fare,
       pay.Payment_Status
FROM Booking b
JOIN Passenger p ON p.Passenger_ID = b.Passenger_ID
JOIN Flight    f ON f.Flight_ID    = b.Flight_ID
LEFT JOIN Ticket t ON t.Booking_ID = b.Booking_ID
LEFT JOIN Payment pay ON pay.Booking_ID = b.Booking_ID
WHERE t.Ticket_ID IS NULL
ORDER BY b.Booking_ID;
```

Booking_ID	Passenger	Flight_Number	Booking_Status	Fare	Payment_Status
5008	Mohammed Faizan	AI502	Pending	5900.00	Pending
5022	Lakshmi Narayanan	6E310	Pending	3200.00	Pending
5037	Deepa Menon	IX771	Pending	3300.00	Pending

Explanation: Finds records that are incomplete. These three pending bookings have an unpaid (Pending) payment and so no ticket; staff could follow them up.

Q5. Passengers with the same surname travelling on the same flight (self join)

```
SELECT p1.Last_Name,
       CONCAT(p1.First_Name, ' & ', p2.First_Name) AS Travellers,
       f.Flight_Number, f.Departure_DateTime
FROM Booking b1
JOIN Booking    b2 ON b2.Flight_ID = b1.Flight_ID AND b1.Passenger_ID<
b2.Passenger_ID
JOIN Passenger p1 ON p1.Passenger_ID = b1.Passenger_ID JOIN
Passenger p2 ON p2.Passenger_ID = b2.Passenger_ID
JOIN Flight    f ON f.Flight_ID = b1.Flight_ID
WHERE p1.Last_Name = p2.Last_Name
  AND b1.Booking_Status <> 'Cancelled' AND
  b2.Booking_Status <> 'Cancelled'
ORDER BY p1.Last_Name, f.Departure_DateTime;
```

Last_Name	Travellers	Flight_Number	Departure_DateTime
Singh	Vikram & Pooja	SG145	2026-11-01 08:00:00
Singh	Neha Vikram & Aditya	SG146	2026-11-01 11:30:00
Singh	Neha & Pooja	AI502	2026-11-02 06:00:00
Verma	Vikram & Neha	6E204	2026-11-01 09:30:00
Verma	Neha Aditya & Pooja	6E205	2026-11-01 12:30:00

Explanation: A self join joins a table to itself (Booking as b1 and b2, Passenger as p1 and p2). The condition b1.Passenger_ID < b2.Passenger_ID stops a pair appearing twice. The result identifies likely families travelling together, useful for seating them side by side.

3. Reports using joins and aggregates

Q6. Flight load report: seats sold against capacity

```
SELECT f.Flight_Number,
       CONCAT(dep.Airport_Code, ' -> ', arr.Airport_Code) AS Route,
       f.Departure_DateTime, a.Aircraft_Model, a.Total_Seats,
       COUNT(b.Booking_ID) AS Booked,
       ROUND(100.0 * COUNT(b.Booking_ID) / a.Total_Seats, 2) AS
Load_Factor_Pct
FROM Flight f
JOIN Aircraft a      ON a.Aircraft_ID = f.Aircraft_ID
JOIN Airport dep ON dep.Airport_ID = f.Departure_Airport_ID
JOIN Airport arr ON arr.Airport_ID = f.Arrival_Airport_ID
JOIN Booking b       ON b.Flight_ID = f.Flight_ID AND b.Booking_Status =
'Confirmed'
GROUP BY f.Flight_ID, f.Flight_Number, dep.Airport_Code, arr.Airport_Code,
         f.Departure_DateTime, a.Aircraft_Model, a.Total_Seats
ORDER BY Load_Factor_Pct DESC, f.Flight_ID
LIMIT 8;
```

Flight_Number	Route	Departure_DateTime	Aircraft_Model	Total_Seats	Booked	Load_Factor_Pct
IX771	BLR -> COK	2026-11-01 07:30:00	ATR 72-600	72	2	2.78

Flight_Number	Route	Departure_DateTime	Aircraft_Model	Total_Seats	Booked	Load_Factor_Pct
AI502	BLR -> DEL	2026-11-01 06:00:00	Airbus A320neo	180	4	2.22
UK821	BOM -> DEL	2026-11-01 07:15:00	Airbus A321	222	4	1.80
AI503	DEL -> BLR	2026-11-01 10:00:00	Airbus A320neo	180	3	1.67
6E204	BLR -> BOM	2026-11-01 09:30:00	Airbus A320neo	180	3	1.67
AI934	BLR -> DXB	2026-11-01 02:30:00	Airbus A320neo	180	3	1.67
SG145	DEL -> CCU	2026-11-01 08:00:00	Airbus A320neo	186	3	1.61
AI502	BLR -> DEL	2026-11-02 06:00:00	Airbus A320neo	180	2	1.11

Explanation: Load factor (seats sold divided by seats available) is a key airline measure. The small ATR 72 shows the highest value. Percentages are low overall because the sample holds only a few bookings per flight.

Q7. Revenue report by route

```
SELECT CONCAT(dep.Airport_Code, ' -> ', arr.Airport_Code) AS Route,
       COUNT(DISTINCT b.Booking_ID) AS Paid_Bookings,
       SUM(pay.Amount)              AS Revenue,
       ROUND(AVG(pay.Amount), 2)    AS Avg_Payment
FROM   Payment
pay  JOIN  Booking  ON b.Booking_ID   = pay.Booking_ID
b JOIN Flight f     ON f.Flight_ID    = b.Flight_ID
JOIN Airport dep ON dep.Airport_ID = f.Departure_Airport_ID
JOIN Airport arr ON arr.Airport_ID = f.Arrival_Airport_ID
WHERE pay.Payment_Status = 'Successful'
  AND b.Booking_Status = 'Confirmed'
GROUP BY dep.Airport_Code, arr.Airport_Code
ORDER BY Revenue DESC
LIMIT 6;
```

Route	Paid_Bookings	Revenue	Avg_Payment
BLR -> DEL	8	64700.00	8087.50
BLR -> SIN	2	58100.00	29050.00
BLR -> DXB	3	52600.00	17533.33
BOM -> DEL	5	51600.00	10320.00
DXB -> BLR	2	39700.00	19850.00
DEL -> BOM	2	27100.00	13550.00

Explanation: Money received comes from Payment, the route from Flight and Airport, so four tables are joined. Only successful payments on confirmed bookings are counted, so the failed retry and pending payments do not inflate revenue.

Q8. Revenue by airline code

```
SELECT SUBSTR(f.Flight_Number, 1, 2) AS Airline_Code,
       COUNT(DISTINCT f.Flight_ID)       AS Flights_With_Sales,
       COUNT(DISTINCT b.Booking_ID) AS Bookings,
       SUM(pay.Amount)                   AS Revenue
FROM Flight f
JOIN Booking b       ON b.Flight_ID       = f.Flight_ID AND b.Booking_Status=
'Confirmed'
JOIN Payment pay ON pay.Booking_ID = b.Booking_ID AND pay.Payment_Status=
'Successful'
GROUP BY SUBSTR(f.Flight_Number, 1, 2)
ORDER BY Revenue DESC;
```

Airline_Code	Flights_With_Sales	Bookings	Revenue
AI	8	19	239000.00
UK	3	7	78700.00
6E	5	10	50200.00
SG	2	5	38400.00
IX	1	2	7150.00

Explanation: The first two characters of a flight number identify the airline. Grouping on a string function gives revenue per airline code.

Q9. Top spenders with their baggage activity

```
SELECT p.Passenger_ID,
       CONCAT(p.First_Name, ' ', p.Last_Name) AS Passenger,
       COUNT(DISTINCT b.Booking_ID) AS Bookings,
       SUM(pay.Amount)              AS Total_Paid,
       SUM(g.Baggage_Count)         AS Bags,
       SUM(g.Excess_Baggage_Fee)    AS Excess_Fees_Paid
FROM Passenger p
JOIN Booking b       ON b.Passenger_ID = p.Passenger_ID AND b.Booking_Status=
'Confirmed'
JOIN Payment pay ON pay.Booking_ID = b.Booking_ID AND pay.Payment_Status=
'Successful'
LEFT JOIN Baggage g ON g.Booking_ID = b.Booking_ID
GROUP BY p.Passenger_ID, p.First_Name, p.Last_Name
ORDER BY Total_Paid DESC
LIMIT 5;
```

Passenger_ID	Passenger	Bookings	Total_Paid	Bags	Excess_Fees_Paid
1	Aarav Patel	3	81800.00	8	0.00
8	Rohan Mehta	3	55400.00	6	0.00
1	Ishita Gupta	2	34000.00	4	6	4000.00
4	Rahul Sharma	3	24400.00	4	3300.00
1	Priya Nair	3	32400.00		900.00

5

Explanation: A customer-value report combining Passenger, Booking, Payment and Baggage. The airline can use it for loyalty offers.

Q10. Airport traffic report (subqueries in FROM)

```
SELECT a.Airport_Code, a.City,
       COALESCE(d.Departures, 0) AS Departures,
       COALESCE(r.Arrivals, 0)   AS Arrivals,
       COALESCE(d.Departures, 0) + COALESCE(r.Arrivals, 0) AS Total_Movements
FROM Airport a
LEFT JOIN (SELECT Departure_Airport_ID AS Airport_ID, COUNT(*) AS Departures
           FROM Flight GROUP BY Departure_Airport_ID) d ON d.Airport_ID =
a.Airport_ID
LEFT JOIN (SELECT Arrival_Airport_ID AS Airport_ID, COUNT(*) AS Arrivals
           FROM Flight GROUP BY Arrival_Airport_ID) r   ON r.Airport_ID =
a.Airport_ID
ORDER BY Total_Movements DESC, a.Airport_Code;
```

Airport_Code	City	Departures	Arrivals	Total_Movements
BLR	Bengaluru	14	7	21
DEL	Delhi	7	6	13
BOM	Mumbai	3	3	6
CCU	Kolkata	1	2	3
COK	Kochi	1	2	3
DXB	Dubai	1	2	3
GOI	Goa	0	2	2
HYD	Hyderabad	0	2	2
MAA	Chennai	1	1	2
SIN	Singapore	0	1	1
AMD	Ahmedabad	0	0	0
PNQ	Pune	0	0	0

Explanation: Departures and arrivals are counted separately in two derived tables (subqueries in the FROM clause) and then joined to Airport. LEFT JOIN with COALESCE shows airports with no flights as 0 instead of dropping them (AMD and PNQ).

4. Subqueries

Q11. Bookings priced above the average confirmed fare (scalar subquery)

```
SELECT Booking_ID, Passenger_ID, Flight_ID, Fare
FROM Booking
WHERE Booking_Status = 'Confirmed'
    AND Fare > (SELECT AVG(Fare) FROM Booking WHERE Booking_Status=
'Confirmed')
ORDER BY Fare DESC, Booking_ID;
```

Booking_ID	Passenger_ID	Flight_ID	Fare
5047	18	128	38500.00
5041	14	121	22000.00
5027	18	112	21800.00
5023	18	111	21500.00
5038	14	120	21500.00
5048	2	128	17800.00
5042	15	121	15200.00
5039	15	120	14800.00
5040	16	120	14800.00
5009	9	103	14500.00
5003	3	101	13800.00
5024	19	111	12800.00
5032	8	114	12400.00
5017	14	107	11900.00

Explanation: The inner query returns one value (the average confirmed fare, 9316.28), which the outer query compares each fare against. The average is not hard-coded, so the report stays correct when data changes.

Q12. Passengers who have booked a Business or First class seat (IN)

```
SELECT Passenger_ID, First_Name, Last_Name, Email
FROM Passenger
WHERE Passenger_ID IN (SELECT b.Passenger_ID
                       FROM Booking b
                       JOIN Seat s ON s.Seat_ID = b.Seat_ID
                       WHERE s.Seat_Class IN ('Business', 'First')
                         AND b.Booking_Status = 'Confirmed')
ORDER BY Passenger_ID;
```

Passenger_ID	First_Name	Last_Name	Email
3	Priya	Nair	priya.nair@example.com
8	Mohammed	Faizan	mohammed.faizan@example.com
9	Sneha	Kulkarni	sneha.kulkarni@example.com
14	Rohan	Mehta	rohan.mehta@example.com
18	Aarav	Patel	aarav.patel@example.com
19	Fatima	Sheikh	fatima.sheikh@example.com

Explanation: The subquery returns a list of passenger IDs and IN keeps the passengers on that list. Each passenger appears once even if they booked premium seats several times.

Q13. Passengers who have flown on an international route (EXISTS)

```
SELECT p.Passenger_ID, CONCAT(p.First_Name, ' ', p.Last_Name) AS Passenger
FROM Passenger p
WHERE EXISTS (SELECT 1
              FROM Booking b
              JOIN Flight f            ON f.Flight_ID            = b.Flight_ID
              JOIN Airport dep ON dep.Airport_ID = f.Departure_Airport_ID
              JOIN Airport arr ON arr.Airport_ID = f.Arrival_Airport_ID
              WHERE b.Passenger_ID = p.Passenger_ID
                AND b.Booking_Status = 'Confirmed'
                AND (dep.Country <> 'India' OR arr.Country <> 'India'))
ORDER BY p.Passenger_ID;
```

Passenger_ID	Passenger
2	Rahul Sharma
14	Rohan Mehta
15	Ishita Gupta
16	Suresh Babu
18	Aarav Patel

Explanation: A correlated subquery: it refers to p.Passenger_ID from the outer query and runs once per passenger. EXISTS is true as soon as one matching booking is found. This is a target list for international offers.

Q14. Bookings with no successful payment (NOT EXISTS)

```
SELECT b.Booking_ID, b.Passenger_ID, b.Booking_Status, b.Fare
FROM Booking b
WHERE NOT EXISTS (SELECT 1
                  FROM Payment pay
                  WHERE pay.Booking_ID = b.Booking_ID
                    AND pay.Payment_Status = 'Successful')
ORDER BY b.Booking_ID;
```

Booking_ID	Passenger_ID	Booking_Status	Fare
5008	8	Pending	5900.00
5022	17	Pending	3200.00
5037	11	Pending	3300.00

Explanation: NOT EXISTS returns bookings for which the subquery finds nothing: money still owed. Booking 5004 is not listed because, although its first payment failed, its retry succeeded.

Q15. Each passenger's most expensive confirmed booking (correlated subquery with MAX)

```
SELECT b.Passenger_ID,
       CONCAT(p.First_Name, ' ', p.Last_Name) AS Passenger,
       b.Booking_ID, b.Flight_ID, b.Fare
FROM Booking b
JOIN Passenger p ON p.Passenger_ID = b.Passenger_ID
WHERE b.Booking_Status = 'Confirmed'
  AND b.Fare = (SELECT MAX(b2.Fare)
                FROM Booking b2
                WHERE b2.Passenger_ID = b.Passenger_ID
                AND b2.Booking_Status = 'Confirmed')
ORDER BY b.Fare DESC, b.Booking_ID
LIMIT 8;
```

Passenger_ID	Passenger	Booking_ID	Flight_ID	Fare
18	Aarav Patel	5047	128	38500.00
14	Rohan Mehta	5041	121	22000.00
2	Rahul Sharma	5048	128	17800.00
15	Ishita Gupta	5042	121	15200.00
16	Suresh Babu	5040	120	14800.00
9	Sneha Kulkarni	5009	103	14500.00
3	Priya Nair	5003	101	13800.00
19	Fatima Sheikh	5024	111	12800.00

Explanation: For every booking the inner query finds that same passenger's highest fare, and only the matching booking is kept. This answers 'largest row per group', which GROUP BY alone cannot.

Q16. Flights busier than the average flight (nested subqueries)

```
SELECT f.Flight_Number, f.Departure_DateTime, t.Bookings
FROM (SELECT Flight_ID, COUNT(*) AS Bookings
        FROM Booking
        GROUP BY Flight_ID) t
JOIN Flight f ON f.Flight_ID = t.Flight_ID
WHERE t.Bookings > (SELECT AVG(x.c)
                    FROM (SELECT COUNT(*) AS c
                          FROM Booking
                          GROUP BY Flight_ID) x)
ORDER BY t.Bookings DESC, f.Flight_ID;
```

Flight_Number	Departure_DateTime	Bookings
AI502	2026-11-01 06:00:00	5
6E204	2026-11-01 09:30:00	4
UK821	2026-11-01 07:15:00	4
AI502	2026-11-02 06:00:00	3
AI503	2026-11-01 10:00:00	3
SG145	2026-11-01 08:00:00	3
IX771	2026-11-01 07:30:00	3
AI934	2026-11-01 02:30:00	3

Explanation: The innermost query counts bookings per flight; the middle one averages those counts (50 bookings over 20 flights = 2.5); the outer query keeps flights above the average. Derived tables (t and x) must have aliases.

Q17. Share of revenue per payment method (subquery in SELECT)

```
SELECT Payment_Method,
        SUM(Amount) AS Collected,
        ROUND(100.0 * SUM(Amount) /
                (SELECT SUM(Amount) FROM Payment WHERE Payment_Status=
'Successful'), 2) AS Share_Pct
FROM Payment
WHERE Payment_Status = 'Successful'
GROUP BY Payment_Method
ORDER BY Collected DESC;
```

Payment_Method	Collected	Share_Pct
Card	179600.00	41.64
Net Banking	161550.00	37.45
UPI	90200.00	20.91

Explanation: The scalar subquery gives the grand total once, so each method's total can be shown as a percentage of it. The three shares add up to 100.

5. Management summary

Q18. One-row management dashboard

```
SELECT (SELECT COUNT(*) FROM Passenger)                                 AS
Passengers,
        (SELECT COUNT(*) FROM Flight)                                   AS
Flights,
        (SELECT COUNT(*) FROM Booking)                                  AS
Bookings,
        (SELECT SUM(Amount) FROM PaymentWHEREPayment_Status='Successful')
AS Revenue_Collected,
        (SELECT ROUND(100.0 * SUM(CASEWHENBooking_Status='Cancelled'THEN
1 ELSE 0 END)
                        / COUNT(*), 2)FROMBooking)                      AS
Cancellation_Rate_Pct;
```

Passengers	Flights	Bookings	Revenue_Collected	Cancellation_Rate_Pct
20	28	50	431350.00	8.00

Explanation: Several scalar subqueries side by side build a summary of the whole system in one row. The cancellation rate uses CASE inside SUM to count only cancelled bookings (4 of 50 = 8%). Revenue_Collected counts every successful payment, including those on bookings that were later cancelled (the sample has no refund status).

6. Joins versus subqueries

	Join	Subquery
Purpose	Combine columns from several tables into one result row	Use the result of one query inside another
Best when	The output needs columns from more than one table (manifests, itineraries, revenue by route)	A value, list or yes/no test is needed for filtering (above-average fares, premium travellers, unpaid bookings)
Finding missing records	LEFT JOIN … IS NULL	NOT EXISTS or NOT IN
Typical risk	Duplicate rows if the join is not 1:1, so aggregates need care	Correlated subqueries can be slower on very large tables

Many questions can be written either way (Q3 could use NOT EXISTS, Q12 could use a join). The choice depends on whether the answer needs columns from the second table and on readability.

7. Conclusion

Joins let the nine normalized tables be recombined into complete records and summaries, such as a flight manifest, a passenger's full itinerary, route and airline revenue, and airport traffic. Subqueries answer questions that depend on another calculation, such as fares above the average, passengers with no successful payment, or flights busier than the

average flight. Used together with GROUP BY and aggregate functions, they turn the Airline Reservation System database into a source of useful operational and management reports.

SQL Code

```sql
-- Q1. Passenger manifest for flight AI502 on 1 November (INNER JOIN, 5 tables)
-- An INNER JOIN returns only rows that match in every table. Airport is joined twice under two aliases (dep and arr) because Flight has two foreign keys to it. This is the manifest for the flight; the cancelled booking 5005 still appears, so a boarding list would add WHERE b.Booking_Status = 'Confirmed'.
SELECT b.Booking_ID,
       CONCAT(p.First_Name, ' ', p.Last_Name) AS Passenger,
       s.Seat_Number, s.Seat_Class,
       CONCAT(dep.Airport_Code, ' -> ', arr.Airport_Code) AS Route,
       b.Booking_Status, b.Fare
FROM  Booking  b
JOIN Passenger p ON p.Passenger_ID = b.Passenger_ID
JOIN    Seat    ON s.Seat_ID      = b.Seat_ID
JOIN   Flight   f ON f.Flight_ID   = b.Flight_ID
JOIN  Airport  dep ON dep.Airport_ID = f.Departure_Airport_ID
JOIN Airport   arr ON arr.Airport_ID = f.Arrival_Airport_ID
WHERE f.Flight_ID = 101
ORDER BY s.Seat_Class, s.Seat_Number;

-- Q2. Complete travel record of one passenger across all nine tables
-- Gathers every record related to passenger 18 (Aarav Patel). INNER JOINs follow the mandatory relationships; LEFT JOINs are used for Ticket, Payment and Baggage because a booking may not have them yet, and the booking must still appear.
SELECT CONCAT(p.First_Name, ' ', p.Last_Name) AS Passenger,
       f.Flight_Number, f.Departure_DateTime,
       CONCAT(dep.Airport_Code, ' -> ', arr.Airport_Code) AS Route,
       a.Aircraft_Model, s.Seat_Number, s.Seat_Class,
       t.Ticket_Number, t.Ticket_Status,
       pay.Payment_Method, pay.Amount, pay.Payment_Status,
       g.Baggage_Count, g.Total_Weight
FROM Passenger p
 JOIN Booking b   ON b.Passenger_ID = p.Passenger_ID
JOIN Flight    f   ON f.Flight_ID    = b.Flight_ID
  JOIN Aircraft a   ON a.Aircraft_ID   = f.Aircraft_ID
JOIN Airport dep ON dep.Airport_ID = f.Departure_Airport_ID
JOIN Airport arr ON arr.Airport_ID = f.Arrival_Airport_ID
JOIN Seat      s    ON s.Seat_ID      = b.Seat_ID
LEFT JOIN Ticket t      ON t.Booking_ID   = b.Booking_ID
LEFT JOIN Payment pay ON pay.Booking_ID = b.Booking_ID AND pay.Payment_Status = 'Successful'
LEFT JOIN Baggage g  ON g.Booking_ID  = b.Booking_ID
WHERE p.Passenger_ID = 18
ORDER BY f.Departure_DateTime;

-- Q3. Flights that nobody has booked (LEFT JOIN ... IS NULL)
```

```sql
-- A LEFT JOIN keeps every flight, filling the Booking columns with NULL when no booking exists. Filtering on NULL then leaves only flights with no bookings: candidates for promotion or cancellation.
SELECT f.Flight_ID, f.Flight_Number, f.Departure_DateTime,
       CONCAT(dep.Airport_Code, '->', arr.Airport_Code) AS Route,
       f.Flight_Status
FROM Flightf
JOIN Airportdep ON dep.Airport_ID = f.Departure_Airport_ID
JOIN Airportarr ON arr.Airport_ID = f.Arrival_Airport_ID
LEFT JOIN Bookingb ON b.Flight_ID = f.Flight_ID
WHERE b.Booking_ID IS NULL
ORDERBYf.Departure_DateTime, f.Flight_ID;

-- Q4.Bookingsthathavenoticketyet,withtheirpaymentstatus
-- Finds  recordsthatareincomplete.Thesethreependingbookingshavean unpaid (Pending) payment and so no ticket; staff could follow them up.
SELECT b.Booking_ID,
       CONCAT(p.First_Name, '', p.Last_Name) AS Passenger,
       f.Flight_Number, b.Booking_Status, b.Fare,
       pay.Payment_Status
FROM Booking b
JOIN Passengerp ON p.Passenger_ID = b.Passenger_ID
JOIN Flight     f ON f.Flight_ID      = b.Flight_ID
LEFT JOIN Ticket t     ON t.Booking_ID = b.Booking_ID
LEFT JOIN Paymentpay ON pay.Booking_ID = b.Booking_ID
WHERE t.Ticket_ID IS NULL
ORDERBYb.Booking_ID;

-- Q5. Passengers with the same surname travelling on the same flight (self join)
-- A self join joins a table to itself (Booking as b1 and b2, Passenger as p1 and p2). The condition b1.Passenger_ID < b2.Passenger_ID stops a pair appearing twice. The result identifies likely families travelling together, useful for seating them side by side.
SELECT p1.Last_Name,
       CONCAT(p1.First_Name, '&', p2.First_Name) AS Travellers,
       f.Flight_Number, f.Departure_DateTime
FROM Booking b1
JOIN Booking  b2 ON b2.Flight_ID = b1.Flight_ID AND b1.Passenger_ID < b2.Passenger_ID
JOIN      Passengerp1     ON      p1.Passenger_ID      = b1.Passenger_ID       JOIN       Passengerp2       ON p2.Passenger_ID = b2.Passenger_ID
JOIN Flight f ON f.Flight_ID = b1.Flight_ID
WHERE p1.Last_Name = p2.Last_Name
  AN  b1.Booking_Statu  <  'Cancelled'
  D   s                 >  'Cancelled'
ORDERBYp1.Last_Name, f.Departure_DateTime;
  D   s                 >

-- ###################### REPORTS USING JOINS AND AGGREGATES ######################

-- Q6. Flight load report: seats sold against capacity
```

```sql
-- Load factor (seats sold divided by seats available) is a key airline measure. The small ATR 72 shows the highest value. Percentages are low overall because the sample holds only a few bookings per flight.
SELECT f.Flight_Number,
       CONCAT(dep.Airport_Code, ' -> ', arr.Airport_Code) AS Route,
       f.Departure_DateTime, a.Aircraft_Model, a.Total_Seats,
       COUNT(b.Booking_ID) AS Booked,
       ROUND(100.0 * COUNT(b.Booking_ID) / a.Total_Seats, 2) AS Load_Factor_Pct
FROM Flight f
JOIN Aircraft a      ON a.Aircraft_ID = f.Aircraft_ID
JOIN Airport dep ON dep.Airport_ID = f.Departure_Airport_ID
JOIN Airport arr ON arr.Airport_ID = f.Arrival_Airport_ID
JOIN Booking b     ON b.Flight_ID = f.Flight_ID AND b.Booking_Status = 'Confirmed'
GROUP BY f.Flight_ID, f.Flight_Number, dep.Airport_Code, arr.Airport_Code,
         f.Departure_DateTime, a.Aircraft_Model, a.Total_Seats
ORDER BY Load_Factor_Pct DESC, f.Flight_ID
LIMIT 8;

-- Q7. Revenue report by route
-- Money received comes from Payment, the route from Flight and Airport, so four tables are joined. Only successful payments on confirmed bookings are counted, so the failed retry and pending payments do not inflate revenue.
SELECT CONCAT(dep.Airport_Code, ' -> ', arr.Airport_Code) AS Route,
       COUNT(DISTINCT b.Booking_ID) AS Paid_Bookings,
       SUM(pay.Amount)              AS Revenue,
       ROUND(AVG(pay.Amount), 2)    AS Avg_Payment
FROM   Payment pay
       JOIN Booking  b   ON b.Booking_ID  = pay.Booking_ID
       JOIN Flight   f   ON f.Flight_ID   = b.Flight_ID
       JOIN Airport dep ON dep.Airport_ID = f.Departure_Airport_ID
       JOIN Airport arr ON arr.Airport_ID = f.Arrival_Airport_ID
WHERE pay.Payment_Status = 'Successful'
  AND b.Booking_Status = 'Confirmed'
GROUP BY dep.Airport_Code, arr.Airport_Code
ORDER BY Revenue DESC
LIMIT 6;

-- Q8. Revenue by airline code
-- The first two characters of a flight number identify the airline. Grouping on a string function gives revenue per airline code.
SELECT SUBSTR(f.Flight_Number, 1, 2) AS Airline_Code,
       COUNT(DISTINCT f.Flight_ID)   AS Flights_With_Sales,
       COUNT(DISTINCT b.Booking_ID)  AS Bookings,
       SUM(pay.Amount)               AS Revenue
FROM Flight f
JOIN Booking b   ON b.Flight_ID    = f.Flight_ID AND b.Booking_Status = 'Confirmed'
JOIN Payment pay ON pay.Booking_ID = b.Booking_ID AND pay.Payment_Status = 'Successful'
GROUP BY SUBSTR(f.Flight_Number, 1, 2)
ORDER BY Revenue DESC;
```

```sql
-- Q9. Top spenders with their baggage activity
-- A customer-value report combining Passenger, Booking, Payment and Baggage. The airline can use it for loyalty offers.
SELECT p.Passenger_ID,
       CONCAT(p.First_Name, ' ', p.Last_Name) AS Passenger,
       COUNT(DISTINCT b.Booking_ID)           AS Bookings,
       SUM(pay.Amount)                        AS Total_Paid,
       SUM(g.Baggage_Count)                   AS Bags,
       SUM(g.Excess_Baggage_Fee)              AS Excess_Fees_Paid
FROM Passenger p
JOIN Booking b   ON b.Passenger_ID = p.Passenger_ID AND b.Booking_Status = 'Confirmed'
JOIN Payment pay ON pay.Booking_ID = b.Booking_ID AND pay.Payment_Status = 'Successful'
LEFT JOIN Baggage g ON g.Booking_ID = b.Booking_ID
GROUP BY p.Passenger_ID, p.First_Name, p.Last_Name
ORDER BY Total_Paid DESC
LIMIT 5;

-- Q10. Airport traffic report (subqueries in FROM)
-- Departures and arrivals are counted separately in two derived tables (subqueries in the FROM clause) and then joined to Airport. LEFT JOIN with COALESCE shows airports with no flights as 0 instead of dropping them (AMD and PNQ).
SELECT a.Airport_Code, a.City,
       COALESCE(d.Departures, 0) AS Departures,
       COALESCE(r.Arrivals, 0)   AS Arrivals,
       COALESCE(d.Departures, 0) + COALESCE(r.Arrivals, 0) AS Total_Movements
FROM Airport a
LEFT JOIN (SELECT Departure_Airport_ID AS Airport_ID, COUNT(*) AS Departures
           FROM Flight GROUP BY Departure_Airport_ID) d ON d.Airport_ID = a.Airport_ID
LEFT JOIN (SELECT Arrival_Airport_ID AS Airport_ID, COUNT(*) AS Arrivals
           FROM Flight GROUP BY Arrival_Airport_ID) r  ON r.Airport_ID = a.Airport_ID
ORDER BY Total_Movements DESC, a.Airport_Code;


-- ###################### SUBQUERIES ######################

-- Q11. Bookings priced above the average confirmed fare (scalar subquery) -- The inner query returns one value (the average confirmed fare, 9316.28), which the outer query compares each fare against. The average is not hard- coded, so the report stays correct when data changes.
SELECT Booking_ID, Passenger_ID, Flight_ID, Fare
FROM Booking
WHERE Booking_Status = 'Confirmed'
  AND Fare > (SELECT AVG(Fare) FROM Booking WHERE Booking_Status = 'Confirmed')
ORDER BY Fare DESC, Booking_ID;

-- Q12. Passengers who have booked a Business or First class seat (IN)

```

```sql
-- The subquery returns a list of passenger IDs and IN keeps the passengers on that list. Each passenger appears once even if they booked premium seats several times.
SELECT Passenger_ID, First_Name, Last_Name, Email
FROM Passenger
WHERE Passenger_ID IN (SELECT b.Passenger_ID
                       FROM Booking b
                       JOIN Seat s ON s.Seat_ID = b.Seat_ID
                       WHERE s.Seat_Class IN ('Business', 'First')
                         AND b.Booking_Status = 'Confirmed')
ORDER BY Passenger_ID;

-- Q13. Passengers who have flown on an international route (EXISTS)
-- A correlated subquery: it refers to p.Passenger_ID from the outer query and runs once per passenger. EXISTS is true as soon as one matching booking is found. This is a target list for international offers.
SELECT p.Passenger_ID, CONCAT(p.First_Name, ' ', p.Last_Name) AS Passenger
FROM Passenger p
WHERE EXISTS (SELECT 1
              FROM Booking b
              JOIN Flight   f   ON f.Flight_ID      = b.Flight_ID
              JOIN Airport dep ON dep.Airport_ID = f.Departure_Airport_ID
              JOIN Airport arr ON arr.Airport_ID = f.Arrival_Airport_ID
              WHERE b.Passenger_ID = p.Passenger_ID
                AND b.Booking_Status = 'Confirmed'
                AND (dep.Country <> 'India' OR arr.Country <> 'India'))
ORDER BY p.Passenger_ID;

-- Q14. Bookings with no successful payment (NOT EXISTS)
-- NOT EXISTS returns bookings for which the subquery finds nothing: money still owed. Booking 5004 is not listed because, although its first payment failed, its retry succeeded.
SELECT b.Booking_ID, b.Passenger_ID, b.Booking_Status, b.Fare
FROM Booking b
WHERE NOT EXISTS (SELECT 1
                  FROM Payment pay
                  WHERE pay.Booking_ID = b.Booking_ID
                    AND pay.Payment_Status = 'Successful')
ORDER BY b.Booking_ID;

-- Q15. Each passenger's most expensive confirmed booking (correlated subquery with MAX)
-- For every booking the inner query finds that same passenger's highest fare, and only the matching booking is kept. This answers 'largest row per group', which GROUP BY alone cannot.
SELECT b.Passenger_ID,
       CONCAT(p.First_Name, ' ', p.Last_Name) AS Passenger,
       b.Booking_ID, b.Flight_ID, b.Fare
FROM Booking b
JOIN Passenger p ON p.Passenger_ID = b.Passenger_ID
WHERE b.Booking_Status = 'Confirmed'
  AND b.Fare = (SELECT MAX(b2.Fare)
                FROM Booking b2
                WHERE b2.Passenger_ID = b.Passenger_ID
```

```
                    AND b2.Booking_Status = 'Confirmed')
ORDER BY b.Fare DESC, b.Booking_ID
LIMIT 8;

-- Q16. Flights busier than the average flight (nested subqueries)
-- The innermost query counts bookings per flight; the middle one averages
those counts (50 bookings over 20 flights = 2.5); the outer query keeps
flights above the average. Derived tables (t and x) must have aliases.
SELECT f.Flight_Number, f.Departure_DateTime, t.Bookings
FROM (SELECT Flight_ID, COUNT(*) AS Bookings
      FROM Booking
      GROUP BY Flight_ID) t
JOIN Flight f ON f.Flight_ID = t.Flight_ID
WHERE t.Bookings > (SELECT AVG(x.c)
                    FROM (SELECT COUNT(*) AS c
                          FROM Booking
                          GROUP BY Flight_ID) x)
ORDER BY t.Bookings DESC, f.Flight_ID;

-- Q17. Share of revenue per payment method (subquery in SELECT)
-- The scalar subquery gives the grand total once, so each method's total can
be shown as a percentage of it. The three shares add up to 100.
SELECT Payment_Method,
       SUM(Amount) AS Collected,
       ROUND(100.0 * SUM(Amount) /
             (SELECT SUM(Amount) FROM Payment WHERE Payment_Status =
'Successful'), 2) AS Share_Pct
FROM Payment
WHERE Payment_Status = 'Successful'
GROUP BY Payment_Method
ORDER BY Collected DESC;


-- ###################### MANAGEMENT SUMMARY
######################

-- Q18. One-row management dashboard
-- Several scalar subqueries side bysidebuildasummaryofthewholesystem in one
row. The cancellation rate usesCASEinsideSUMtocountonlycancelled bookings (4
of 50 = 8%). Revenue_Collectedcountseverysuccessfulpayment, including those
on bookings that werelatercancelled(thesamplehasno refund status).
SELECT (SELECT COUNT(*) FROM Passenger)                                AS
Passengers,
       (SELECT COUNT(*) FROM Flight)                                   AS
Flights,
       (SELECT COUNT(*) FROM Booking)                                  AS
Bookings,
       (SELECT SUM(Amount) FROM Payment WHERE Payment_Status = 'Successful')
AS Revenue_Collected,
       (SELECT ROUND(100.0 * SUM(CASE WHEN Booking_Status = 'Cancelled' THEN 1
ELSE 0 END)
                     / COUNT(*), 2) FROM Booking)                      AS
Cancellation_Rate_Pct;
```